

产品需求文档 (PRD): SwarmOS 协同开发平台 (MVP V1.0)

一、产品背景与一期目标 (Background & Objectives)

系统旨在打破传统 AI 代码助手（如单一 Claude Code）无法实现复杂任务切分、容易把本地物理环境跑崩的痛点。

本期 MVP 核心目标：跑通“人类下发需求 → 后端引擎拆解 → 本地端执行与锁控”的闭环，实现零代码界面的 AI 自动改代码与测试。

二、系统边界与架构约束 (System Boundaries)

为了保证独立开发和稳定运行，系统强制拆分为三个模块，各自承担单一职责：

- 展现层 (UI)**：只负责渲染群聊界面、AI 思考日志卡片和操作审批按钮。**禁止**包含任何大模型调用和 Agent 逻辑。
- 调度中台 (Middleware)**：系统的唯一大脑。负责接收前端指令、动态实例化 Agent，并将任务通过 WebSocket 下发给物理机。
- 执行终端 (Daemon)**：运行在开发者物理机上的静默进程。只负责接收指令执行文件读写、跑命令，并强制执行文件并发锁。

三、核心功能需求 (Functional Requirements)

3.1 动态智能体工厂 (Dynamic Agent Factory)

- 需求描述**：中台必须抛弃读取本地静态 YAML 文件的设定，支持通过接口接收 JSON 参数，在内存中动态组装并启动 Agent（配置系统提示词、绑定的模型和允许使用的工具）。
- 前置依赖**：Ruflo 核心调度引擎。

3.2 思考链路实时流传输 (Thought Stream Interception)

- 需求描述**：Agent 在执行复杂规划、工具调用前的内部推导过程，不能隐藏在后端控制台中。必须将其捕获，并以流式（Stream）实时推送到前端展现层。
- 交互规则**：前端接收到此类消息时，不作为普通聊天气泡展示，需渲染在一个可折叠的“思考状态面板”中。

3.3 物理端高并发文件锁 (Local File Mutex Lock)

- **需求描述：** 当平台中存在多个 Agent 同时工作时，极易发生覆盖写代码的情况。本地执行终端 (Daemon) 必须实现排他锁机制。
- **业务规则：**
 - a. 当 Agent-A 申请修改 `/src/api.js` 并获得批准后，该文件进入 `Locked` 状态。
 - b. 此时若 Agent-B 申请读取或修改该文件，Daemon 拒绝执行并返回状态 `waiting_lock`。
 - c. 调度中台收到拒绝信号后，将 Agent-B 挂起，放入重试队列，直到锁释放。

3.4 终端防自爆与强杀 (Timeout Kill Switch)

- **需求描述：** 防止 AI 写出死循环代码导致本地 CPU 满载。
- **业务规则：** 任何终端命令 (如 `npm run test` 或 python 脚本) 执行时间超过 **60 秒**，Daemon 必须强制终止该子进程，并向中台抛出 `TimeoutError` 日志。

3.5 高危操作的人工阻断审批 (Human-in-the-Loop)

- **需求描述：** AI 员工执行敏感操作前必须获得人类授权。
- **业务规则：**
 - a. 预设高危动作名单 (如 `git push`, `rm -rf`, 数据库写操作)。
 - b. 中台拦截到高危动作调用时，暂停 Agent 进程，向前端发送审批请求。
 - c. 前端渲染 `[Approve]` 与 `[Reject]` 按钮。用户点击后，中台放行或取消该动作。

四、核心数据字典与通信协议 (Data Protocols)

所有上下游模块间的数据通信，均采用标准 JSON 格式。

4.1 接口协议：创建虚拟蜂群 (HTTP POST)

- **路由：** `/api/v1/swarm/init` (由调度中台提供)
- **Payload 规范：**
- **JSON**

代码块

```
1  {
2    "channel_id": "c_9901",
3    "agents": [
4      {
5        "agent_id": "coder_main",
6        "role": "Senior Developer",
```

```

7      "model": "claude-3-5-sonnet",
8      "system_prompt": "你负责处理具体的代码修改...",
9      "allowed_tools": ["mcp_read", "mcp_write"]
10     }
11   ]
12 }

```

4.2 消息协议：推送思考日志 (WebSocket: 中台 → UI)

- **Payload 规范：**

JSON

代码块

```

1      {
2        "event_type": "thought_log",
3        "agent_id": "coder_main",
4        "content": "检测到目标文件，正在调用读取工具...",
5        "timestamp": 1718000000
6      }
7    ``
8
9    ### 4.3 动作指令协议：下发物理执行任务 (WebSocket: 中台 → Daemon)
10   *   **请求 Payload (中台下发): **
11
12   ```json
13     {
14       "action_id": "act_889",
15       "tool": "file_write",
16       "target_path": "/app/main.js",
17       "params": { "content": "console.log('hello');" }
18     }
19   ```
20   *   **响应 Payload (Daemon 结果回传): **
21
22   ```json
23     {
24       "action_id": "act_889",
25       "status": "success", // 或者 "error", "waiting_lock"
26       "stdout_log": "文件保存成功"
27     }
28   ```
29   ---
30
31   ## 五、 MVP (V1.0) 验收标准 (Acceptance Criteria)

```

32
33 研发期结束时，必须满足以下所有标准才算通过验收：
34
35 1. **连通性验收：** 调度中台服务启动后，本地电脑上的 Daemon 能够建立稳定连接并接收控制。
36 2. **安全性验收：** 使用两个脚本模拟向 Daemon 发送同一文件的并发修改请求，系统不报错且能按照先来后到的顺序执行。
37 3. **全链路业务验收：** 在前端 UI 框中输入“在本地创建一个 utils.py，写一个冒泡排序，然后执行它”。系统无需人类介入，能在 2 分钟内自动完成文件创建、代码编写、执行，并将最终执行结果展示在前端群聊面板中。